

001 - Bootcamp (Kanban) Deployment Reference Guide

Infrastructure: DigitalOcean VPS (Ubuntu) + Docker Compose + Caddy + Watchtower **Registry:** GitHub Container Registry (GHCR) **Domain:** kanban.lazuar.com

1. Generate a GitHub Personal Access Token (PAT)

Both GitHub Actions and your VPS need permission to read/write your Docker images on GHCR. You must generate a PAT from the account that has repository access.

1. Go directly to: <https://github.com/settings/tokens>
 2. Click **Generate new token** -> **Generate new token (classic)**.
 3. Set the **Note** (e.g., "GHCR Bootcamp Deployment").
 4. Set **Expiration** to "No expiration" (or 1 year if preferred).
 5. Under **Select scopes**, check the following boxes:
 - **write:packages** (this will auto-select **read:packages** and **repo**)
 6. Scroll to the bottom and click **Generate token**.
 7. **COPY THE TOKEN IMMEDIATELY** (it starts with **ghp_...**). You will never see it again.
-

2. Configure GitHub Actions Permissions

For GitHub Actions to successfully build and push, it needs **Read and write permissions**. Because this repository belongs to an organization (**proxeon**), you must enable this at the Organization level *before* you can enable it at the Repository level.

Step 2.1: Organization Level

1. Go to: <https://github.com/organizations/proxeon/settings/actions>
2. Scroll down to **Workflow permissions**.
3. Select **Read and write permissions**.
4. Click **Save**.

Step 2.2: Repository Level

1. Go to: <https://github.com/proxeon/bootcamp/settings/actions>
 2. Scroll down to **Workflow permissions**.
 3. Select **Read and write permissions**.
 4. Click **Save**.
-

3. Configure GitHub Secrets

For the CI/CD pipeline to push images securely, you need to add your credentials to the GitHub Repository Secrets.

1. Go to your repository on GitHub -> **Settings** -> **Secrets and variables** -> **Actions**.
2. Click **New repository secret** and add the following two secrets:

Name	Value Example	Description
CR_USER	allaboutevemirolive	The exact GitHub username that generated the PAT
CR_PAT	ghp_xxxx...	The Personal Access Token you generated in Step 1

4. Link GHCR Packages to the Repository (Crucial)

If your Docker images were ever pushed manually (e.g., from your local machine), GitHub locks the package permissions. You **must** grant the repository explicit **Write** access to the packages so GitHub Actions isn't blocked with a **403 Forbidden** error.

1. Go to the Organization Packages page: <https://github.com/orgs/proxeon/packages>
2. Click on the **bootcamp/api** package.
3. On the right sidebar, click **Package Settings** (or go to <https://github.com/orgs/proxeon/packages/container/bootcamp%2Fapi/settings>).
4. Scroll down to **Manage Actions access**.
5. Click **Add repository**, search for **bootcamp**, and add it.
6. Change the Role dropdown from **Read** to **Write**.
7. **REPEAT** this exact process for the **bootcamp/web** and **bootcamp/admin** packages.

5. DNS Configuration (Cloudflare)

Before deploying, ensure your DNS record is set. Initially, disable the Cloudflare Proxy (Orange Cloud -> Grey Cloud) to allow Caddy to issue Let's Encrypt SSL certificates.

Type	Name	Content	Proxy Status
A	kanban	[VPS_IP_ADDRESS]	DNS Only (Grey)

(Note: Because we are using path-based routing like **/api** and **/admin**, you only need one single DNS record!)

6. Server Configuration (VPS)

SSH into your DigitalOcean VPS. Ensure Docker and UFW (Firewall) are configured to allow port 80 and 443.

6.1. Directory Structure

Create a directory to hold the production configuration.

```
mkdir -p ~/kanban-prod
cd ~/kanban-prod
```

6.2. Authentication

Login to GitHub Container Registry on the VPS so Watchtower can pull your private images.

```
export GH_USERNAME="your-github-username"
export CR_PAT="ghp_your_token_here"

echo $CR_PAT | docker login ghcr.io -u $GH_USERNAME --password-stdin
```

7. Deployment Files (On VPS)

Create the following three files inside `~/kanban-prod`.

File 1: `.env`

This file contains runtime secrets and configurations. Replace the `REPO_USER` and `REPO_PASS` with your GitHub username and the PAT.

```
# Database (DigitalOcean Managed PostgreSQL)
DATABASE_URL=Host=db-
postgresql-...;Port=25060;Database=defaultdb;Username=doadmin;Password=...;
SSL Mode=Require;Trust Server Certificate=true

# Admin Seed
ADMIN_EMAIL=admin@kanban.local
ADMIN_PASSWORD=Admin123!
ADMIN_DISPLAY_NAME=System Admin

# ASP.NET
ASPNETCORE_ENVIRONMENT=Production

# CORS & Cookies
CORS_ORIGINS=https://kanban.lazuar.com
COOKIE_DOMAIN=kanban.lazuar.com

# Watchtower (GitHub Credentials for automated updates)
REPO_USER=your-github-username
REPO_PASS=ghp_your_token_here
```

File 2: `Caddyfile`

This configures the reverse proxy and automatic SSL.

```
kanban.lazuar.com {
    # API Traffic -> .NET API
    handle /api/* {
        reverse_proxy api:5010
    }
}
```

```

# SignalR Hub (WebSocket support)
handle /api/hubs/* {
    reverse_proxy api:5010
}

# Admin Panel -> Admin Caddy Container
handle /admin/* {
    reverse_proxy admin:80
}

# Everything Else -> Next.js Web App
handle /* {
    reverse_proxy web:3000
}
}

```

File 3: `docker-compose.yml`

This orchestrates the containers. Notice how we map `ghcr.io/proxeon/bootcamp/...` — if you reuse this setup on another repo, remember to change that path to match your new repo.

```

services:
  # --- GATEWAY (Caddy - HTTPS + Reverse Proxy) ---
  gateway:
    image: caddy:2-alpine
    container_name: kanban-gateway
    restart: always
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile
      - caddy_data:/data
      - caddy_config:/config
    depends_on:
      - api
      - web
      - admin

  # --- API (.NET) ---
  api:
    image: ghcr.io/proxeon/bootcamp/api:latest
    container_name: kanban-api
    restart: always
    environment:
      - DATABASE_URL=${DATABASE_URL}
      - ASPNETCORE_ENVIRONMENT=${ASPNETCORE_ENVIRONMENT}
      - ADMIN_EMAIL=${ADMIN_EMAIL}
      - ADMIN_PASSWORD=${ADMIN_PASSWORD}
      - ADMIN_DISPLAY_NAME=${ADMIN_DISPLAY_NAME}
      - CORS_ORIGINS=${CORS_ORIGINS}

```

```
    - COOKIE_DOMAIN=${COOKIE_DOMAIN}

# --- WEB (Next.js) ---
web:
  image: ghcr.io/proxeon/bootcamp/web:latest
  container_name: kanban-web
  restart: always

# --- ADMIN (Vite + Caddy) ---
admin:
  image: ghcr.io/proxeon/bootcamp/admin:latest
  container_name: kanban-admin
  restart: always

# --- WATCHTOWER (Auto-Update from GHCR) ---
watchtower:
  image: containrrr/watchtower
  container_name: kanban-watchtower
  restart: always
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock
  environment:
    - REPO_USER=${REPO_USER}
    - REPO_PASS=${REPO_PASS}
    - DOCKER_API_VERSION=1.40
  command: --interval 300 --cleanup

volumes:
  caddy_data:
  caddy_config:
```

8. Launch & Verification

Run these commands inside the `~/kanban-prod` folder on the VPS.

1. Start Services:

```
docker compose up -d
```

2. Verify Running Containers:

```
docker compose ps
```

Ensure all containers (`gateway`, `api`, `web`, `admin`, `watchtower`) are listed with State "Up".

3. Check SSL Generation:

```
docker compose logs --tail=50 -f gateway
```

Look for "certificate obtained successfully".

4. **Check Application Health:** Visit <https://kanban.lazuar.com> in your browser.

9. Standard Update Procedure (CI/CD)

Because **GitHub Actions** and **Watchtower** are configured, the update process is now fully automated!

1. Make code changes on your local machine.
 2. Push your changes to the `main` branch: `git push origin main`
 3. GitHub Actions will automatically build and push the new images to GHCR.
 4. Within ~5 minutes, Watchtower will detect the new images, pull them to the VPS, and seamlessly restart the updated containers without you ever having to SSH into the server!
-

10. Manual Build Process (Fallback / Local Testing)

If you ever need to manually build and push images from your local machine, use the following commands. Ensure you have run `docker login ghcr.io` first using your PAT.

```
export GH_ORG="proxeon"
export GH_REPO="bootcamp"

# 1. Backend API
docker buildx build --platform linux/amd64 --push \
  -t ghcr.io/$GH_ORG/$GH_REPO/api:latest \
  -f services/api-dotnet/Dockerfile .

# 2. Web App (Next.js)
docker buildx build --platform linux/amd64 --push \
  -t ghcr.io/$GH_ORG/$GH_REPO/web:latest \
  -f apps/web/Dockerfile .

# 3. Admin App (Vite)
docker buildx build --platform linux/amd64 --push \
  -t ghcr.io/$GH_ORG/$GH_REPO/admin:latest \
  -f apps/admin/Dockerfile .
```